



Agenda

Form the smallest number from single digits

Count Sort

Merge two sorted arrays

Merge Sort

Stable & Inplace Sort

The Contest 1: Arrays, Bit Manipulation, Recursion, Math & Hashing i

Find the **smallest number** that can be formed by **rearranging the digits** of the given number in an array. Return the **smallest number** in the form an array.

ans $\rightarrow$ { 1, 2, 2, 3, 4, 6, 7 }

number 0-9 only

A = { 4, 2, 7, 3, 9, 0 }

{ 0, 2, 3, 4, 7, 9 }

Idea $\rightarrow$ Sorting will take $N \log N \rightarrow$ reduce TC? $\rightarrow$ $O(N)$

A = { 6, 3, 4, 2, 7, 1, 1 } $\leftarrow$

To create freq array loop on original array.

0	1	2	3	4	5	6	7	8	9
0	1	2	1	1	0	1	1	0	0

for $i \rightarrow 0$ to 9
 for $j \rightarrow 1$ to $A[i]$
 $A[idx] = i$
 $idx++$

i = 0	j =	idx
0	0	0.
1	1	1
2	1	1 $\rightarrow$ 2
2	2	3.

0 2 2 2 4 5, 6

A = { 1, 2, 2, 3, 4, 6, 7 }

This is count sort

$N \rightarrow$ no of 11ps

TC: $O(N)$

SC: $O(1)$

Size of array is 10^9

1 int $\rightarrow$ 4 byte

4 byte $\times 10^9 \rightarrow$ 4GB

Note $\rightarrow$ Count sort is good only for number upto 10^6 .
used for small number.

$\rightarrow$ Negative Numbers

$A = [-2, 3, 8, 3, -2, 3]$

min = -2, max = 8.

$$\begin{aligned} [a, b] &= b - a + 1 \\ &= 8 - (-2) + 1 \\ &= 8 + 2 + 1 \\ &= 11 \end{aligned}$$

$\rightarrow -2, -1, 0, 1, 2, 3, 4, 5, 6, 7, 8$
 $0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10$

$$\begin{aligned} -2 + 2 &= 0 \\ 3 + 2 &= 5 \\ 8 + 2 &= 10 \\ 3 + 2 &= 5 \\ -2 + 2 &= 0 \end{aligned}$$

$A[i] - (\text{smallest_number})$

Code $\rightarrow$

```
calculate  
min; max  
f[max-min+1]  
for (i  $\rightarrow$  0 to N) {  
    idx = A[i] - min  
    f[idx] = f[idx] + 1;  
}
```

$\rightarrow$ least the freq array ✓

Scenario

Google's Gmail offers an "All Inboxes" feature that allows users to view emails from multiple email accounts in one seamless interface. This is particularly useful for users managing personal and professional communications through separate accounts.

The feature ensures that emails from all accounts are merged into a single feed sorted by date and time, facilitating better email management and access.

Problem

Develop a function to emulate the "All Inboxes" feature of Gmail.

You are given two sorted arrays that represent timestamps of emails from two different email accounts. Each element in the array is an email object.

Your task is to merge these two arrays into a single list, ensuring that the resulting list is sorted by the timestamp, allowing the user to view emails in a chronological order from both accounts combined.

$$\begin{array}{l} \xrightarrow{\text{Sorted } \checkmark} \\ A = \{ 1, 5, 6, 9 \} \\ B = \{ 2, 4, 8 \} \\ \xrightarrow{\text{Sorted } \checkmark} \end{array} \quad \} \text{ merge these sorted arrays}$$

AN: $\{ 1, 2, 4, 5, 6, 8, 9 \}$.

↓

How to merge two sorted arrays.

Given an integer array where all odd elements are sorted and all even elements are sorted.
Sort the entire array.

eN oN
even $\rightarrow 4$, odd $\rightarrow 5$

$A[] = \{2, 5, 4, 8, 11, 13, 10, 15, 21\}$

calculate no of even, odd.

$eIdx \rightarrow 0 \rightarrow 2 \rightarrow 4 \rightarrow 6$ even $idx = \{2, 4, 8, 10\}$
 $oIdx \rightarrow 1 \rightarrow 3 \rightarrow 5 \rightarrow 7$ odd $idx = \{5, 11, 13, 15, 21\}$

5

idx

0, 1, 2, 3, 4, 5, 6, 7, 8

$A = \{2, 5, 4, 8, 11, 13, 10, 15, 21\}$
2 4 5 8 10 11 13 15 21

$idx \rightarrow 0 \times 1 \times 2 \times 3 \times 4 \times 5 \times 6$

$eIdx \rightarrow 0 \times 1 \times 2 \times 3 \times 4$

$oIdx \rightarrow 0 \times 1 \times 2 \times 3 \times 4 \times 5$

even $= \{2, 4, 8, 10\}$

odd $= \{5, 11, 13, 15, 21\}$

while ($eIdx < eN$ $\&\&$ $oIdx < oN$) {

if (even[$eIdx$] < odd[$oIdx$])

$A[eIdx] = even[eIdx];$

$eIdx++;$

else {

$A[oIdx] = odd[oIdx];$

$oIdx++;$

$idx++;$

$T(0, oN)$
 $SC: O(N)$

if even number $4 < 4 \times$

while ($eIdx < eN$) {

$A[eIdx] = even[eIdx];$

$eIdx++;$ $idx++;$

2) odd Number

while ($oIdx < oN$) {

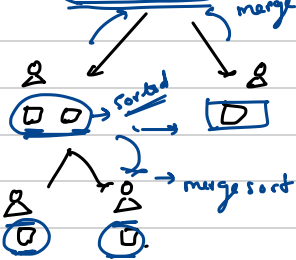
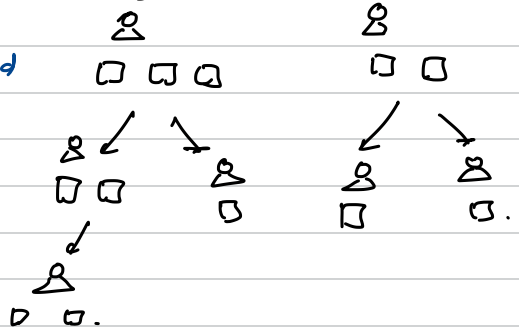
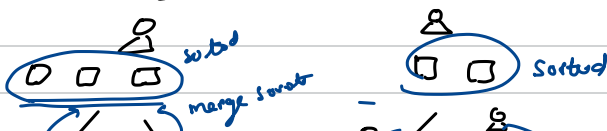
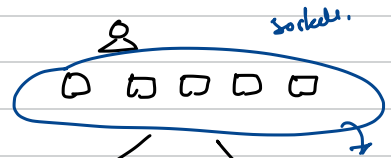
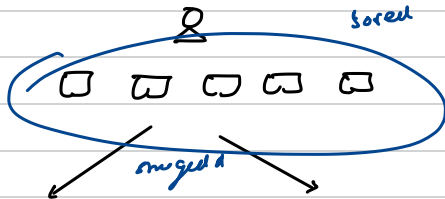
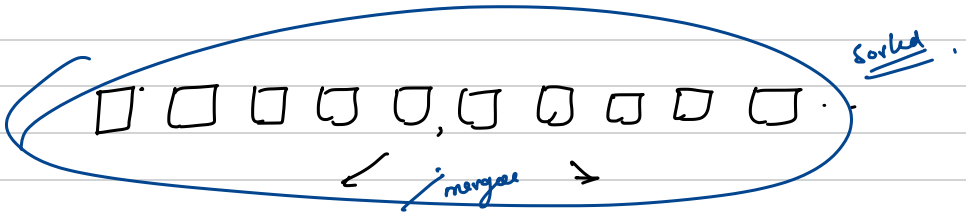
$A[oIdx] = odd[oIdx];$

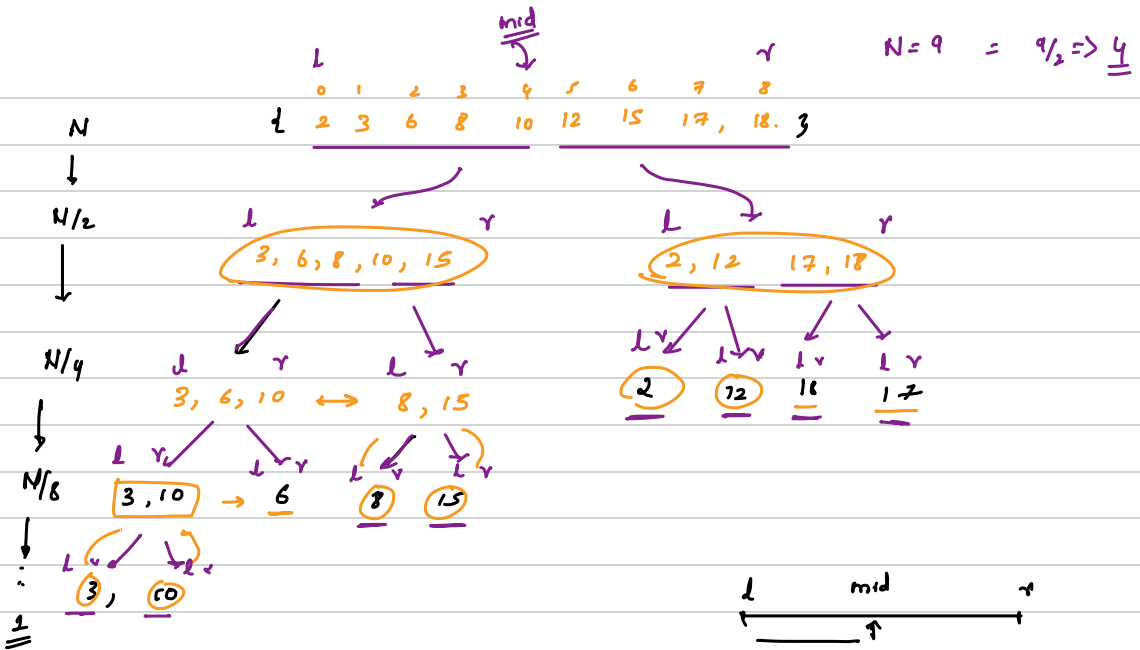
$oIdx++;$ $idx++;$

}

Break →

10:26pm → 10:36pm





```

fun mergeSort(A[], l, r) l
    if (l == r) return r
    mid = (l + r) / 2
    mergeSort(A, l, mid)
    mergeSort(A, mid + 1, r)
    merge(A, l, mid, r)
  
```

$\log N$
 (N)
 $N \log N$

TC: $N \log N$
 SC: $O(N)$

Pseudocode

```
function merge(A[], l, mid, r) {
```

```
    N = A.length();
```

```
    n1 = mid-l+1;
```

```
    n2 = r-mid;
```

```
    B[n1], C[n2];
```

```
    idx=0;
```

```
    for(i -> l to mid){
```

```
        B[idx] = A[i];
```

```
        idx++;
```

```
    }
```

```
    idx=0;
```

```
    for(i -> mid+1 to r){
```

```
        C[idx] = A[i];
```

```
        idx++;
```

```
    }
```

```
    idx = l;
```

```
    i = 0; // moves over A
```

```
    j = 0; // moves over B
```

```
    while (i < n1 && j < n2) {
```

```
        if (B[i] <= C[j]) {
```

```
            A[idx] = B[i];
```

```
            i++;
```

```
        } else {
```

```
            A[idx] = C[j];
```

```
            j++;
```

```
        }
```

```
        idx++;
```

```
    }
```

```
    while (i < n1) {
```

```
        A[idx] = B[i];
```

```
        idx++;
```

```
        i++;
```

```
    }
```

```
    while (j < n2) {
```

```
        A[idx] = C[j];
```

```
        idx++;
```

```
        j++;
```

```
    }
```

```
}
```

```
function mergeSort(A[], l, r){
```

```
    if(l == r) return; // base case
```

```
    mid = (l + r) / 2;
```

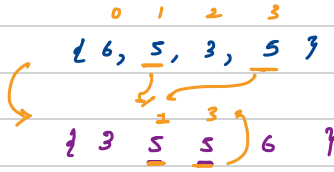
```
    mergeSort (A, l, mid);
```

```
    mergeSort (A, mid + 1, r);
```

```
    merge(A, l, mid, r);
```

```
}
```

Stable Sorting



When a merge happens. $\rightarrow$ if order of original array is marked b/w some.

$\downarrow$
Stable sort

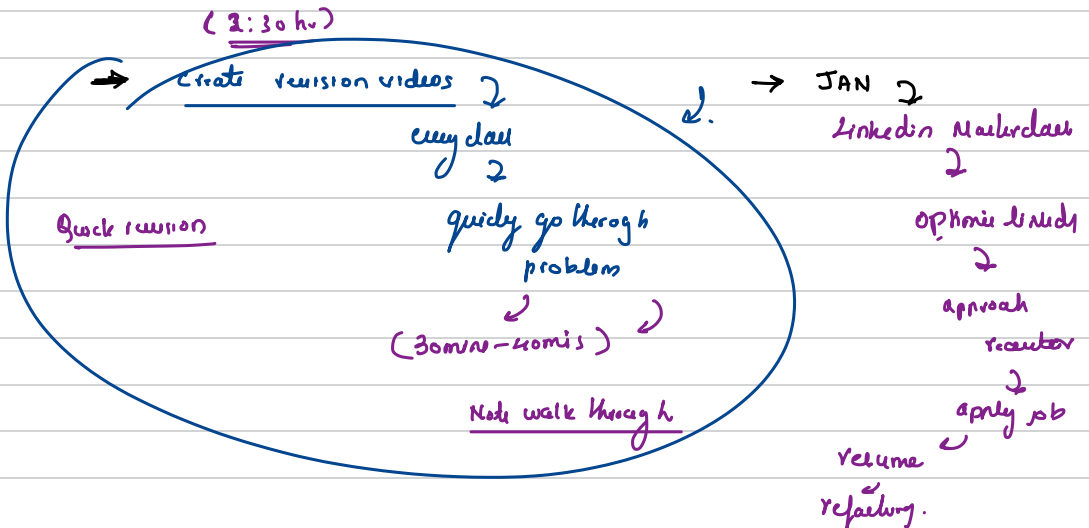
Inplace Sorting

$A = \{1, 5, 4, 3, 2\}$

if we use some array and sort

$\downarrow$
Inplace sorting

$\downarrow$
S.C.C.I.D



2 ← x complete

self motivated
↓

Accountability

have a buddy

4-5 → getting a job.

keep on too.

4 h 5

PSP → 100%

① ② ③
try to solve

explain the
doubt / bug / code.

pushing PSP
→
Control
↓
Mock interview